OrderForge — Architecture & Design Document

Revision 3.1 / 2026-04-15 / Owner: Platform Engineering

1. Document Purpose and Audience

This document describes the internal architecture of the OrderForge service for engineers, operators, and product managers. It is the authoritative reference for design decisions and should be updated whenever those decisions change. Parts of this document duplicate information that appears in the README and the operations runbook on purpose; the goal is that anyone picking up a single document can understand the relevant subset without jumping between sources. When information in this document conflicts with production configuration, production configuration wins and this document is the one that needs updating.

2. Context and Motivation

OrderForge replaced the monolithic legacy-order-backend in late 2024. The old system coupled reservation orders, standard orders, and subscription orders inside a single transaction. A slow payment provider or a single database deadlock would block every path simultaneously. OrderForge splits these concerns so each bounded context can fail and recover independently. The migration took seven months and involved shadow-writing to both systems for the final six weeks until cutover. During that period the two systems had to agree byte-for-byte on the resulting order document, which is why the current storage model looks verbose compared to a greenfield design.

3. Order State Machine

Every order is a finite state machine. Transitions are driven exclusively by domain events; there is no API that directly mutates order state. The canonical state graph is: DRAFT -> PENDING_PAYMENT -> (PAID | EXPIRED), PAID -> PICKING -> PACKED -> SHIPPED -> DELIVERED, DELIVERED -> (RETURN_REQUESTED -> RETURNED). Timeouts for each state are defined in config/state_timeouts.yaml and enforced by a background worker that runs every 30 seconds. The state machine is the single source of truth; any other subsystem reading order state must derive it from here.

4. Bounded Contexts

Five bounded contexts live inside a single deployable. They share a process and a database but not a schema. Ordering owns the state machine and the order aggregate. PricingAdapter is the outbound gateway to the pricing engine. PaymentOrchestrator owns the saga that coordinates multi-method payments. FulfillmentBridge translates carrier webhooks into internal events. Returns handles the post-delivery workflow. The decision to keep them in one deployable rather than splitting into services came down to the cost of distributed transactions; everything we tried added more complexity than it removed.

5. Data Storage

PostgreSQL 15 is the system of record. The schema is partitioned by created_month so that old data can be archived without a full table rewrite. Read replicas serve analytical dashboards but never drive write paths. Redis holds idempotency keys, active order locks, and short-lived caches for customer profile lookups; a Redis failure degrades performance but does not take down order creation. Kafka is the event backbone: every state transition produces a message on the orderforge.events topic and downstream services subscribe. The retention is seven days, which is long enough for any downstream consumer to recover from a multi-day outage.

6. Payment Orchestration — Part One

Payments are the most complex part of the system. A single order can be paid with one card, multiple cards, a combination of points and a card, or a split across a gift certificate and a card. Each method has different idempotency guarantees from the underlying gateway. The orchestrator uses the saga pattern: it builds a plan of authorization calls, executes them in order, and if any step fails it runs the compensation plan to cancel already-authorized amounts. The plan is persisted so that recovery after a process crash can resume exactly where it left off. See section 11 for continuation of this topic.

OrderForge — Architecture & Design Document

Revision 3.1 / 2026-04-15 / Owner: Platform Engineering

7. Inventory Interaction

OrderForge does not own inventory. It sends reservation commands to the inventory service and trusts the reply. Reservation happens before pricing because a sold-out item should fail fast without calling the pricing engine. If the pricing call later fails and we roll back the order, we also release the reservation. Reservations have a TTL on the inventory side so that a crashed OrderForge instance never leaves stock permanently locked; the TTL is set to the configured order expiration plus a thirty second safety margin.

8. Fulfillment Integration

Seven carriers deliver orders today. Each carrier sends webhooks with different payload shapes, authentication schemes, and retry behaviours. The FulfillmentBridge has one adapter per carrier and a single normalized event produced for each incoming webhook. The adapters are intentionally dumb: they translate shape and hand off to the shared state transition code. This means adding a new carrier is a matter of writing a single adapter class and adding the credentials; no core code changes.

9. Returns and Exchanges

A return is modelled as a child entity of the original order. The parent order stays immutable and a return_request row captures the reason, the items being returned, and the financial adjustment. Exchanges are internally a pair of linked return and new order records; the pair is identified by an exchange_pair_id so that reports can collapse the two into a single customer-facing transaction. The return policy limits returns to three separate requests per order and enforces the global 30-day window from delivery.

10. Observability

We emit structured JSON logs on stdout so that the Kubernetes log pipeline can ship them to Elasticsearch without reformatting. Every log record contains trace_id, span_id, order_id (when relevant), and a stable event_code that the alerting rules pattern-match against. Metrics are exposed as Prometheus counters and histograms; the naming follows the orderforge_ prefix convention. Traces flow to Jaeger and are sampled at 5% by default with forced sampling on errors. Dashboards are in Grafana under the orderforge folder.

11. Payment Orchestration — Part Two

Continuing from section 6, the implementation detail worth highlighting is the handling of partial failures. If the first card authorization succeeds for 50,000 KRW but the point deduction for the remaining 10,000 KRW fails, the orchestrator must cancel the first authorization before the customer sees a charge. The cancellation call to the gateway is itself idempotent; we keep retrying until it succeeds or until an operator escalates. In the rare case where the gateway confirms a charge we never asked for, a reconciliation job compares our expected state with the gateway's daily settlement file and flags discrepancies for human review.

12. Security Posture

Customer PII is stored only where strictly required: shipping address, recipient name, recipient phone. Payment details are never stored on our side; we only keep tokens issued by the gateways. All database connections use TLS and the primary cluster sits inside a private VPC. The application runs as a non-root user in a read-only filesystem container. Secrets come from the cluster secret manager at pod start and are rotated every 90 days by an automated job. Network egress is restricted to the payment gateway IP ranges and internal service CIDRs.

OrderForge — Architecture & Design Document

Revision 3.1 / 2026-04-15 / Owner: Platform Engineering

13. Capacity Planning

At peak traffic the service handles about 650 orders per second. The bottleneck on the critical path is the pricing engine call, which caps out around 800 requests per second due to downstream constraints. We mitigate this with a local cache of frequently applied coupons and a circuit breaker that falls back to list prices if the engine starts to degrade. Capacity is reviewed each quarter against campaign plans; the last review raised the minimum replica count from six to eight ahead of the year-end sales period.

14. Testing Strategy

Unit tests cover the domain modules exhaustively and must run in under thirty seconds. Integration tests start real Postgres, Redis, and Kafka containers and run the actual HTTP handlers. Contract tests exchange stubs with pricing and inventory so that a breaking change in their API shows up in our CI rather than in production. Chaos tests run weekly against the staging cluster and inject network partitions, slow dependencies, and message loss to verify our resilience claims.

15. Deployment Topology

Production runs in three regions active-active with primary data residency in Seoul. Each region has its own Postgres primary and Redis cluster, plus a shared global Kafka deployment. Cross-region reads fall back to the local cache if the global cluster is unreachable. Deployments are blue-green: the new version runs in parallel on half the replicas until the error rate over five minutes is within the historical window, then traffic migrates fully. Rollbacks are a single command but operators prefer roll-forward when the issue is not obviously catastrophic.

16. Known Limitations

The service does not currently support partial shipments from a single order; all items must ship together or the order splits into multiple child orders. Multi-tenant isolation is cooperative rather than enforced; a runaway tenant can cause noisy neighbour effects. International shipping requires manual intervention for customs declarations because our templates are only translated for three carriers so far.

17. Upcoming Work

The next major revision will introduce partial shipment support and a declarative fraud rule engine. The shipment work touches the state machine so it is scoped carefully; the fraud engine is additive and expected to ship first. A longer-term item is replacing the saga implementation with an outbox-based approach, which we believe will simplify recovery and reduce the amount of process memory held during payment flows.

18. Appendix A — Glossary

Idempotency-Key: a caller-supplied token that allows safe retries of mutating requests; if a request with a previously-seen key arrives, the cached response is returned rather than executing the action again. Saga: a sequence of local transactions with defined compensations, used here to coordinate multi-step payment flows without a distributed transaction. Bounded Context: a cohesive subset of the domain model with its own ubiquitous language; OrderForge contains five. DLQ: dead letter queue; where events go when they cannot be processed after all retries.